

PE-EC702A — Adaptive Signal Processing

Semester VII • 3L:0T:0P • 3 Credits

Module 4 Exam Notes: Lattice Filters & Linear Prediction

CGEC | B.Tech ECE (2023–27) | *Biraj Sarkar*

Contents

1	Vector Space of Random Variables	2
2	Forward and Backward Projections	2
2.1	Forward Linear Prediction	2
2.2	Backward Linear Prediction	2
3	Stochastic Lattice Filters	2
3.1	Lattice Recursions	3
3.2	Optimal Reflection Coefficients	3
3.3	Single-Stage Lattice Block Diagram	3
4	Relationship with AR Modeling and Levinson-Durbin Recursion	3
4.1	Levinson-Durbin Recursion	4
5	Joint Process Estimator and Gradient Adaptive Lattice	4
5.1	Joint Process Estimator (JPE)	4
5.2	Gradient Adaptive Lattice (GAL)	4

1. Vector Space of Random Variables

Random variables can be modeled as vectors in a Hilbert space H .

- **Inner Product:** For zero-mean random variables X and Y , the inner product is defined as their correlation:

$$\langle X, Y \rangle = \mathbb{E}[XY^*] \quad (1)$$

- **Norm:** The norm corresponds to the root-mean-square (RMS) value:

$$\|X\| = \sqrt{\langle X, X \rangle} = \sqrt{\mathbb{E}[|X|^2]} \quad (2)$$

- **Orthogonality:** Two random variables are orthogonal if $\langle X, Y \rangle = \mathbb{E}[XY^*] = 0$, meaning they are statistically uncorrelated.

2. Forward and Backward Projections

Linear prediction involves estimating a signal sample from adjacent samples.

2.1 Forward Linear Prediction

Predicts the current sample $x(n)$ based on the past M samples $\{x(n-1), \dots, x(n-M)\}$.

- **Forward Prediction Error ($f_M(n)$):**

$$f_M(n) = x(n) - \sum_{k=1}^M a_{M,k}^* x(n-k) \quad (3)$$

where $a_{M,k}$ are the forward prediction coefficients.

2.2 Backward Linear Prediction

Predicts the past sample $x(n-M)$ using the current and subsequent M samples $\{x(n), \dots, x(n-M+1)\}$.

- **Backward Prediction Error ($g_M(n)$):**

$$g_M(n) = x(n-M) - \sum_{k=1}^M b_{M,k}^* x(n-k+1) \quad (4)$$

where $b_{M,k}$ are the backward prediction coefficients. For WSS processes, $b_{M,k} = a_{M,M-k+1}^*$.

3. Stochastic Lattice Filters

Lattice filters represent a modular structure realizing both forward and backward prediction errors order-recursively.

3.1 Lattice Recursions

For a stage m (where $m = 1, 2, \dots, M$):

Lattice Prediction Error Updates

$$f_m(n) = f_{m-1}(n) - \kappa_m^* g_{m-1}(n-1) \quad (5)$$

$$g_m(n) = g_{m-1}(n-1) - \kappa_m f_{m-1}(n) \quad (6)$$

Initial conditions for the recursion are: $f_0(n) = g_0(n) = x(n)$.

Here, κ_m is the **Reflection Coefficient** (PARCOR coefficient) for stage m .

3.2 Optimal Reflection Coefficients

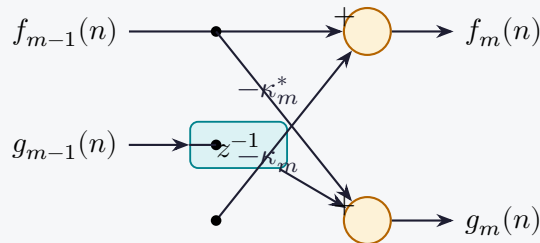
To minimize the sum of the forward and backward mean square errors ($\mathbb{E}[|f_m(n)|^2] + \mathbb{E}[|g_m(n)|^2]$), we differentiate with respect to κ_m , giving:

$$\kappa_m = \frac{2\mathbb{E}[f_{m-1}(n)g_{m-1}^*(n-1)]}{\mathbb{E}[|f_{m-1}(n)|^2] + \mathbb{E}[|g_{m-1}(n-1)|^2]} \quad (7)$$

By Cauchy-Schwarz, $|\kappa_m| \leq 1$. This ensures that the all-pole synthesis filter (inverse lattice filter) is guaranteed to be stable if and only if $|\kappa_m| < 1$ for all m .

3.3 Single-Stage Lattice Block Diagram

Lattice Filter Stage m Block Structure



4. Relationship with AR Modeling and Levinson-Durbin Recursion

An Autoregressive (AR) process of order M is generated by filtering white noise through an all-pole filter. The AR parameters and reflection coefficients are equivalent representations of the same process.

4.1 Levinson-Durbin Recursion

It solves the Yule-Walker equations $\mathbf{R}\mathbf{a} = \mathbf{r}$ with $O(M^2)$ complexity instead of $O(M^3)$ matrix inversion. The recursion steps for order $m = 1, 2, \dots, M$:

$$\Delta_{m-1} = r(m) - \sum_{i=1}^{m-1} a_{m-1,i} r(m-i) \quad (8)$$

$$\kappa_m = \frac{\Delta_{m-1}}{E_{m-1}} \quad (9)$$

$$a_{m,i} = a_{m-1,i} - \kappa_m a_{m-1,m-i}^* \quad (i = 1, \dots, m-1) \quad (10)$$

$$a_{m,m} = \kappa_m \quad (11)$$

$$E_m = E_{m-1}(1 - |\kappa_m|^2) \quad (12)$$

where $E_0 = r(0)$ is the input variance, and κ_m is the reflection coefficient.

5. Joint Process Estimator and Gradient Adaptive Lattice

5.1 Joint Process Estimator (JPE)

The JPE uses the backward prediction errors $g_m(n)$ of a lattice predictor as the input to a transversal filter to estimate a desired signal $d(n)$. Because the backward errors $\{g_0(n), \dots, g_M(n)\}$ are mutually orthogonal, the estimator weights h_m can be optimized independently:

$$\hat{d}(n) = \sum_{m=0}^M h_m^* g_m(n), \quad h_m = \frac{\mathbb{E}[d(n)g_m^*(n)]}{\mathbb{E}[|g_m(n)|^2]} \quad (13)$$

5.2 Gradient Adaptive Lattice (GAL)

GAL is the adaptive implementation of the lattice filter. The reflection coefficients κ_m are updated recursively using a stochastic gradient approach:

$$D_m(n) = \beta D_m(n-1) + (1 - \beta) [|f_m(n)|^2 + |g_m(n-1)|^2] \quad (14)$$

$$\kappa_{m+1}(n+1) = \kappa_{m+1}(n) + \frac{\mu}{D_m(n)} [f_m(n)g_m^*(n-1)] \quad (15)$$

where β is a smoothing factor and $D_m(n)$ estimates the power at stage m .

Example: Levinson-Durbin Recursion

Given autocorrelation values $r(0) = 1.0, r(1) = 0.5, r(2) = 0.125$. Run the Levinson-Durbin recursion to find the reflection coefficients κ_1, κ_2 and the predictor coefficients of a second-order predictor.

Solution: Initialize: $E_0 = r(0) = 1.0$. **Iteration 1** ($m = 1$):

$$\Delta_0 = r(1) = 0.5$$

$$\kappa_1 = \frac{\Delta_0}{E_0} = \frac{0.5}{1.0} = 0.5$$

Predictor coefficient:

$$a_{1,1} = \kappa_1 = 0.5$$

Update error energy:

$$E_1 = E_0(1 - \kappa_1^2) = 1.0(1 - 0.25) = 0.75$$

Iteration 2 ($m = 2$):

$$\Delta_1 = r(2) - a_{1,1}r(1) = 0.125 - (0.5 \times 0.5) = 0.125 - 0.25 = -0.125$$

$$\kappa_2 = \frac{\Delta_1}{E_1} = \frac{-0.125}{0.75} = -\frac{1}{6} \approx -0.1667$$

Predictor coefficients:

$$a_{2,2} = \kappa_2 = -0.1667$$

$$a_{2,1} = a_{1,1} - \kappa_2 a_{1,1} = 0.5 - \left(-\frac{1}{6} \times 0.5\right) = 0.5 + 0.0833 = 0.5833$$

Update error energy:

$$E_2 = E_1(1 - \kappa_2^2) = 0.75 \left(1 - \frac{1}{36}\right) = 0.75 \times \frac{35}{36} = 0.7292$$

The reflection coefficients are $\kappa_1 = 0.5$, $\kappa_2 = -0.1667$, and predictor coefficients are $a_{2,1} = 0.5833$, $a_{2,2} = -0.1667$.

Quick Revision — Module IV

Lattice Filters & Linear Prediction | PE-EC702A

Key Formulas at a Glance

Forward Error	$f_m(n) = f_{m-1}(n) - \kappa_m^* g_{m-1}(n-1)$
Backward Error	$g_m(n) = g_{m-1}(n-1) - \kappa_m f_{m-1}(n)$
Reflection Coeff. (κ_m)	$\kappa_m = \frac{2\mathbb{E}[f_{m-1}(n)g_{m-1}^*(n-1)]}{\mathbb{E}[f_{m-1}(n) ^2] + \mathbb{E}[g_{m-1}(n-1) ^2]}$
Levinson-Durbin κ_m	$\kappa_m = \Delta_{m-1}/E_{m-1}$
Predictor Coefficient Update	$a_{m,i} = a_{m-1,i} - \kappa_m a_{m-1,m-i}^* \quad (a_{m,m} = \kappa_m)$
Error Energy Update	$E_m = E_{m-1}(1 - \kappa_m ^2)$

One-Line Definitions

- **Forward Prediction Error:** The difference between the current signal sample and its linear estimate based on past samples.
- **Backward Prediction Error:** The difference between a past signal sample and its estimate based on subsequent samples.
- **Reflection Coefficient:** The correlation metric κ_m bounded in $[-1, 1]$ parameterizing a lattice stage.
- **Levinson-Durbin Recursion:** An efficient $O(M^2)$ recursive algorithm to solve the Yule-Walker equations.
- **Joint Process Estimator:** A structure utilizing orthogonal backward errors to estimate an external desired signal.
- **GAL Filter:** A gradient-based adaptive lattice filter that adjusts reflection coefficients stage-by-stage.

Frequently Asked / Viva Questions

Q1. What are the initial conditions for the lattice filter recursions?

$f_0(n) = g_0(n) = x(n)$, representing the raw input sample.

Q2. Explain why lattice filters are preferred in hardware over transversal (FIR) filters.

Lattice filters are modular. Increasing the filter order simply requires adding more identical stages at the end, without recalculating prior stages' coefficients.

Q3. What condition must reflection coefficients satisfy to ensure system stability?

$|\kappa_m| < 1$ for all stages m . If any $|\kappa_m| \geq 1$, the corresponding synthesis filter becomes unstable.

Frequently Asked / Viva Questions (contd.)

Q4. Why are backward prediction errors $g_m(n)$ orthogonal to each other?

Because $g_m(n)$ represents the component of $x(n-m)$ that is orthogonal to the subspace spanned by $\{x(n), \dots, x(n-m+1)\}$. Thus, $\mathbb{E}[g_i(n)g_j^*(n)] = 0$ for $i \neq j$.

Q5. How does the computational complexity of Levinson-Durbin compare to direct matrix inversion?

Levinson-Durbin requires $O(M^2)$ operations, whereas direct inversion of the correlation matrix via Gaussian elimination requires $O(M^3)$ operations.

Q6. What is the relationship between predictor coefficients and reflection coefficients?

Predictor coefficients $a_{M,k}$ are calculated from reflection coefficients κ_m using the step-up recursion relation: $a_{m,i} = a_{m-1,i} - \kappa_m a_{m-1,m-i}^*$.

Q7. What is a Joint Process Estimator (JPE)?

It is an adaptive structure that first orthogonalizes the input signal using a lattice filter, and then estimates a desired response by forming a weighted linear combination of the orthogonal backward prediction errors.

Q8. Define the PARCOR coefficient.

It stands for PARTial CORrelation coefficient, which is mathematically identical to the reflection coefficient κ_m in a lattice filter.

Q9. Why does the error energy E_m decrease with increase in predictor order m ?

Because $E_m = E_{m-1}(1 - |\kappa_m|^2)$, and since $|\kappa_m| < 1$, the factor $(1 - |\kappa_m|^2) \leq 1$, meaning adding more taps can only reduce (or keep equal) the prediction error power.

Q10. What is the role of the power estimator $D_m(n)$ in the GAL algorithm?

It normalizes the step-size for each stage individually (similar to NLMS), ensuring that convergence speed is uniform across all stages regardless of local signal power.

Comparison of Transversal vs. Lattice Predictor Structures

Property	Transversal (Tapped Delay Line)	Lattice Structure
Modularity	Low (adding a tap changes all weights)	High (adding a tap is just adding a stage)
Stability Check	Complex (requires finding roots of polynomial)	Simple (checks if $ \kappa_m < 1$ for all stages)
Numerical Sensitivity	High (sensitive to coefficient quantization)	Low (robust to finite word-length effects)
Complexity	$O(M)$ multiplications per update	$O(M)$ multiplications per update (slightly higher constant)